



北京師範大學
BEIJING NORMAL UNIVERSITY

计算机图形学 实验报告

姓名 杨博文 学号 202311081015
院系 人工智能学院 专业 计算机科学与技术

2025 年 5 月 16 日

摘 要

计算机图形学的第七次实验，主要内容包括为三维重建。

目录

1 运行环境及运行方式	3
2 任务 1:300 次运行	3
2.1 任务要求	3
2.2 关键代码	3
2.3 Loss 解释	3
2.4 结果图片	5
3 任务 2:20 个视角	6
3.1 任务要求	6
3.2 关键代码	7
3.3 结果图片	8
4 任务 3: 改变纹理	10
4.1 任务要求	10
4.2 结果	10

1 运行环境及运行方式

硬件为 Macbook Air, M2 芯片, 操作系统为 MacOS Sequoia 15.0。

IDE 为 Pycharm2024.1.3。虚拟环境由 conda 管理。

2 任务 1:300 次运行

2.1 任务要求

在“fit_textured_mesh.py”的基础上, 设置两处的总循环轮数 (Niter) 为 300、可视化结果间隔轮数 (plot_period) 为 50, 运行并保存训练过程中的模型、渲染图像及 loss 结果; 结合教程 notebook 与“modules/pytorch3d/pytorch3d/loss”中的代码, 理解 loss 的设计, 在实验文档中写出各 loss 的计算方法及含义。

2.2 关键代码

优化步骤数

Niter = 300

将优化步骤数改为 300 即可。

2.3 Loss 解释

我们已经将解释写在了源代码的注释中, 现复制如下:

使网格形状平滑/规则化的loss

```
def update_mesh_shape_prior_losses(mesh, loss):
```

```
    # and (b) the edge length of the predicted mesh
```

```
    loss["edge"] = mesh_edge_loss(mesh)
```

```
    """衡量网格中各条边的长度差异
```

```
    防止优化过程中 mesh 被拉伸得过长或压缩过短, 保持拓扑结构合理"""
```

```
    # mesh normal consistency
```

```
    loss["normal"] = mesh_normal_consistency(mesh)
```

```
    """衡量相邻面片之间法线方向的一致性
```

```
    鼓励 mesh 表面光滑, 防止表面出现折痕或突变"""
```

```
    # mesh laplacian smoothing
```

```
    loss["laplacian"] = mesh_laplacian_smoothing(mesh, method="uniform")
```

```
    """鼓励每个顶点接近其邻居的平均值位置
```

鼓励局部平滑，防止局部区域剧烈抖动"""

loss: 使网格形状平滑/规则化的损失

```
loss = {k: torch.tensor(0.0, device=device) for k in losses}
```

```
update_mesh_shape_prior_losses(new_src_mesh, loss)
```

"""这里面含3个loss，在函数定义中解释"""

在每次迭代中随机选择两个视图进行优化。

与仅使用一个视图相比，这有助于解决更新网格形状与更新网格纹理之间的歧义

```
for j in np.random.permutation(num_views).tolist()
```

```
[:num_views_per_iteration]:
```

```
    images_predicted = renderer_textured(new_src_mesh,
```

```
    cameras=target_cameras[j], lights=lights)
```

loss: 从我们的数据集中预测的轮廓和目标轮廓之间的平方L2距离

```
predicted_silhouette = images_predicted[..., 3]
```

```
loss_silhouette = ((predicted_silhouette - target_silhouette[j])
```

```
** 2).mean()
```

```
loss["silhouette"] += loss_silhouette / num_views_per_iteration
```

"""轮廓损失 loss_silhouette

监督预测模型渲染出来的轮廓是否与目标一致

作用：优化网格的几何结构形状，特别是轮廓边缘部分。"""

loss: 预测的RGB图像和我们数据集中的目标图像之间的平方L2距离

```
predicted_rgb = images_predicted[..., :3]
```

```
loss_rgb = ((predicted_rgb - target_rgb[j]) ** 2).mean()
```

```
loss["rgb"] += loss_rgb / num_views_per_iteration
```

"""RGB 图像损失 loss_rgb

对渲染出的 RGB 图像（前 3 通道）与真实图片进行逐像素平方差计算

作用：优化网格的纹理坐标和颜色贴图参数，使生成图像与真实图像接近。"""

loss: 损失加权和

```
sum_loss = torch.tensor(0.0, device=device)
```

```
for k, l in loss.items():
```

```
    sum_loss += l * losses[k] ["weight"]
```

```
    losses[k] ["values"].append(float(l.detach().cpu()))
```

"""最终loss由所有loss加权得到"""

2.4 结果图片

这里我们看一看结果吧 ~

```
tutorials-t1 — yangbowen@Bowen-MacBook-Air — ../tutorials-t1 — -zsh...
iter 190 : total_loss = 0.079251
iter 195 : total_loss = 0.076187
iter 200 : total_loss = 0.067159
iter 205 : total_loss = 0.049077
iter 210 : total_loss = 0.067326
iter 215 : total_loss = 0.059340
iter 220 : total_loss = 0.044443
iter 225 : total_loss = 0.063899
iter 230 : total_loss = 0.072688
iter 235 : total_loss = 0.051875
iter 240 : total_loss = 0.055032
iter 245 : total_loss = 0.068142
iter 250 : total_loss = 0.057048
iter 255 : total_loss = 0.048625
iter 260 : total_loss = 0.063738
iter 265 : total_loss = 0.063337
iter 270 : total_loss = 0.054730
iter 275 : total_loss = 0.052376
iter 280 : total_loss = 0.043541
iter 285 : total_loss = 0.047583
iter 290 : total_loss = 0.057256
iter 295 : total_loss = 0.057188
iter final : total_loss = 0.047303
(FitMesh) yangbowen@Bowen-MacBook-Air tutorials-t1 %
```

图 1: 实验 1 Loss 截图

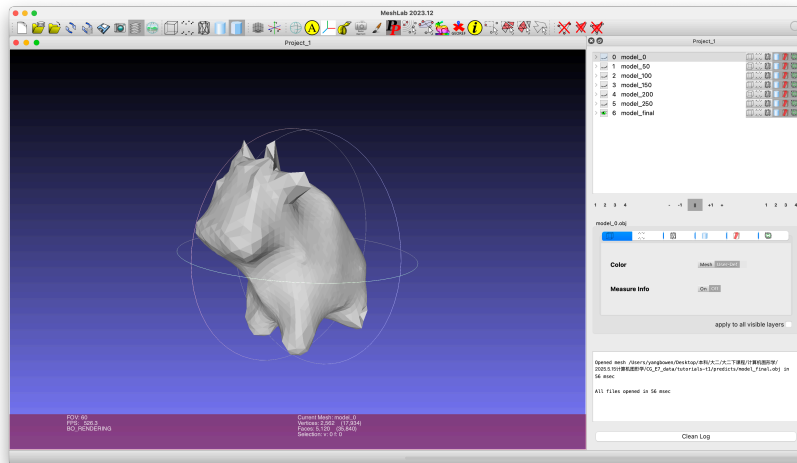


图 2: 实验 1 最终模型

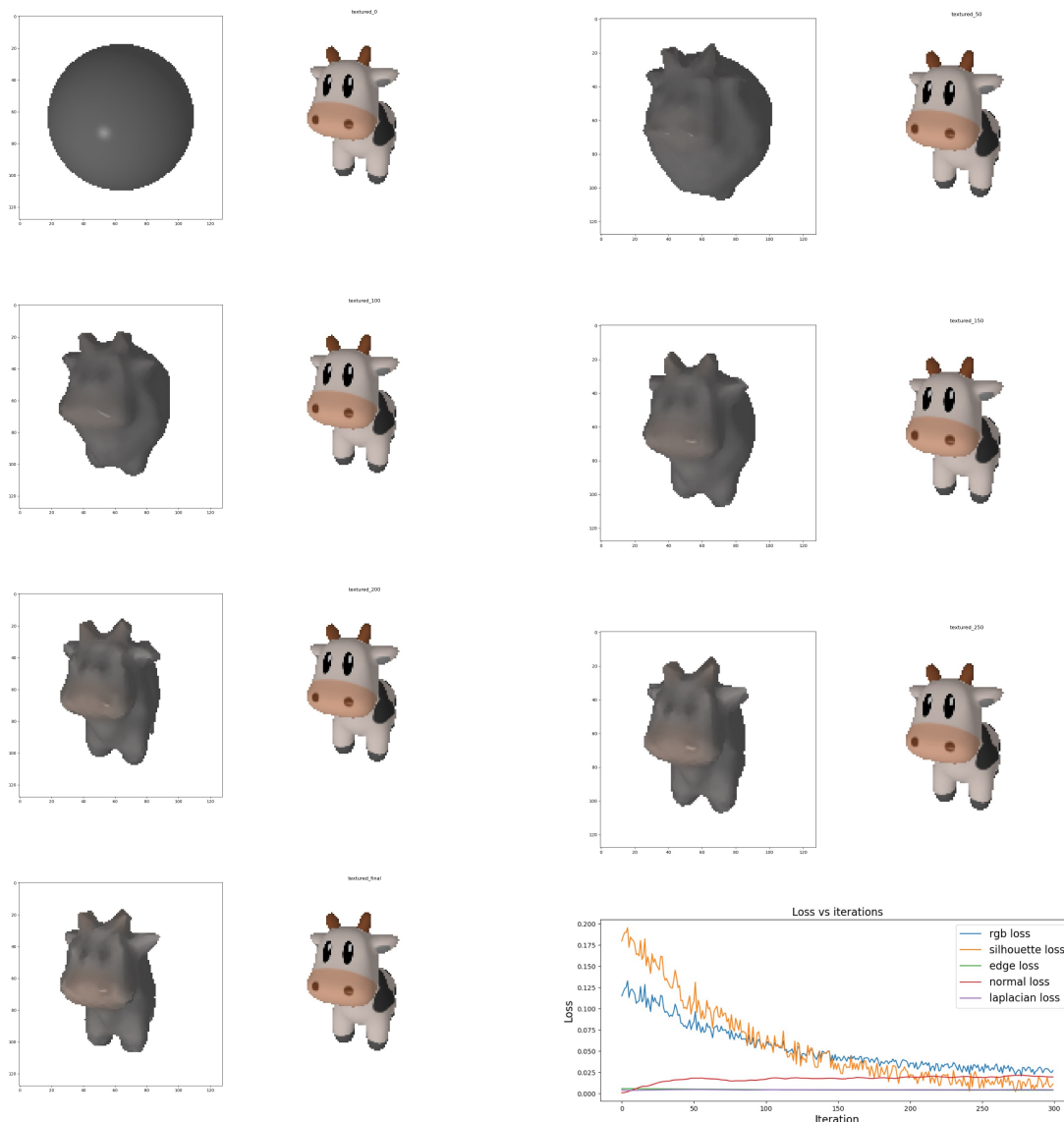


图 3: 实验 1 输出

3 任务 2:20 个视角

3.1 任务要求

在任务 1 的基础上，增加 `visualize_prediction_multiview` 函数，每 50 轮可视化当前重建模型在所有 20 个视角的渲染结果，运行并保存训练过程中的模型、渲染图像及 loss 结果；每次展示的 20 个视角放在单张图像上，不需要对比；

查看 loss 曲线和按时间变化的多视角模型重建结果，在实验文档中分析训练流程，并估计 loss 收敛点，观察该收敛时刻及之后时刻的多视角重建图像/模型，回答并解释“网格模型重建是否在该时刻收敛”。

3.2 关键代码

```
def visualize_prediction_multiview(mesh, renderer, cameras, iteration, outdir="result"):

    os.makedirs(outdir, exist_ok=True)
    images = []

    for cam in cameras:
        with torch.no_grad():
            rendered = renderer(mesh, cameras=cam, lights=lights)
            if silhouette:
                image = rendered[0, ..., 3].cpu().numpy()
                image = np.stack([image]*3, axis=-1) # 转为 RGB
            else:
                image = rendered[0, ..., :3].cpu().numpy()
            images.append(image)

    if len(images) == 0:
        print(f"[Warning] No images rendered at iteration {iteration}")
        return

    # 统一为 numpy 数组
    images_np = np.stack(images, axis=0) # [N, H, W, 3]

    # 自动确定行列数, 最多一行 5 个图
    num_images = images_np.shape[0]
    cols = min(5, num_images)
    rows = (num_images + cols - 1) // cols

    H, W = images_np.shape[1:3]
    grid = np.zeros((rows * H, cols * W, 3), dtype=np.float32)

    for idx, img in enumerate(images_np):
        r = idx // cols
        c = idx % cols
        grid[r*H:(r+1)*H, c*W:(c+1)*W, :] = img

    plt.imsave(f"{outdir}/predicted_multiview_iter_{iteration}.png", np.clip(grid, 0,
```

```
plt.close('all')
...
visualize_prediction_multiview(new_src_mesh, renderer_textured, target_camera)
visualize_prediction_multiview(new_src_mesh, renderer_textured, target_cameras, i)
```

我们写一个生成视角图片的函数，然后调用即可。

3.3 结果图片

这里我们进行 loss 分析：rgb loss 从约 0.13 降至 0.025 左右，下降趋势平稳，在 250 次左右趋于收敛。silhouette loss 起初震荡剧烈（0.2），但下降迅速，在 200 次后趋于稳定，最终也在 0.02 左右。edge loss 从头到尾几乎保持稳定，非常小（ <0.005 ），说明边缘结构没有剧烈变动。normal loss 初期缓慢上升，在约 20 次迭代后趋于稳定（0.02），反映模型法线一致性在收敛初期快速调整。laplacian loss 始终稳定（0.005），说明网格平滑过程比较稳健。

在第 300 轮，轮廓几乎完全拟合真实目标，网格几何也基本完成（牛角、四肢都有），不再出现严重畸变或错误贴图，但颜色依然有一定问题。

已基本收敛。还应该继续进行 100 轮训练看接下来的结果，已确定颜色 loss 是否能继续向下降变得更好。

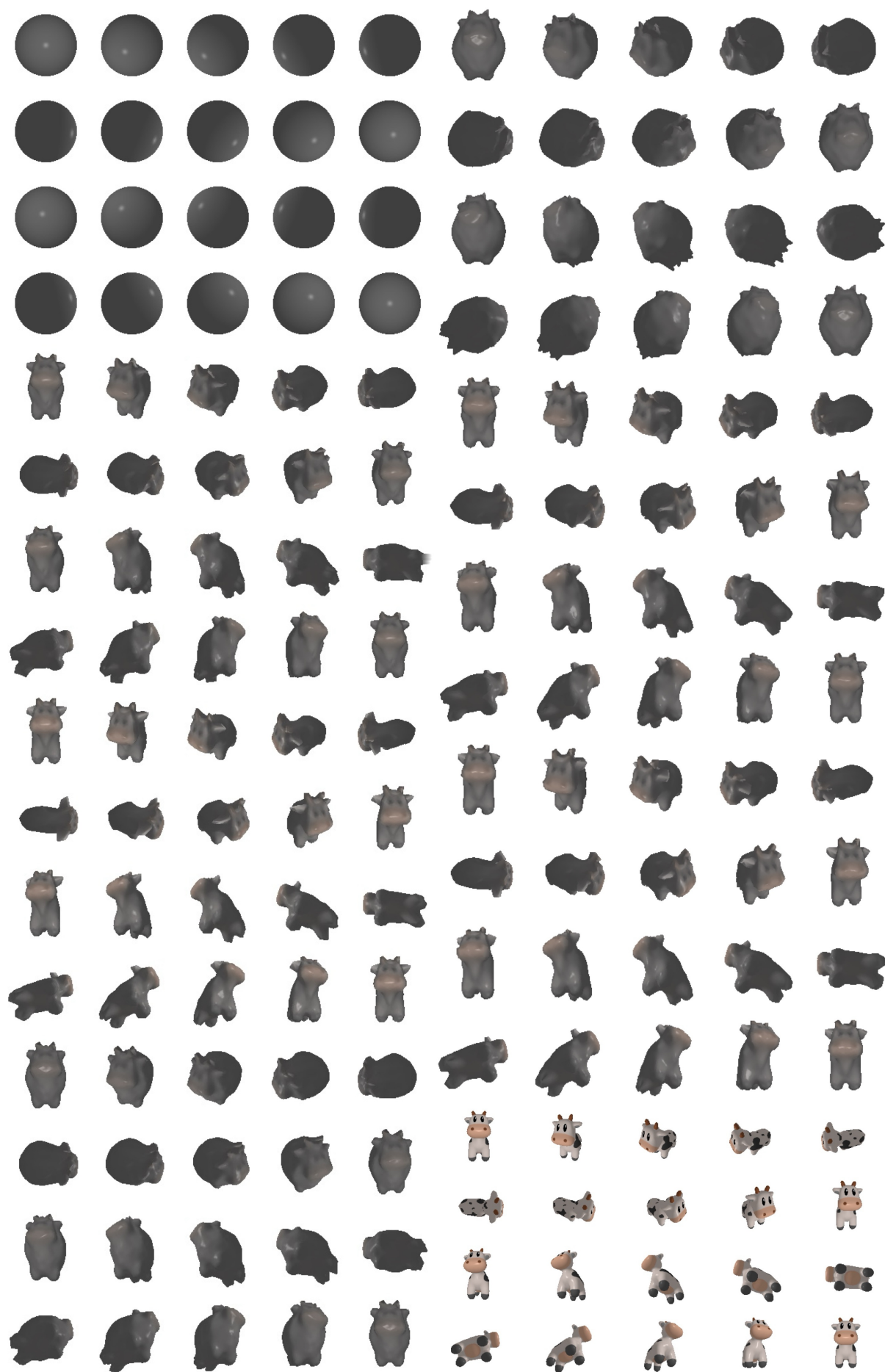


图 4: 实验2 视角图片

4 任务 3: 改变纹理

4.1 任务要求

在任务 2 的基础上，变更模型为更复杂的网格如 backpack，运行并保存训练过程中的模型、渲染图像及 loss 结果；查看对比简单模型与复杂模型的 loss 曲线、直观图像对比，在实验文档中分析两种网格模型的重建结果，回答并解释“实验所用的方法是否能重建复杂模型”。

4.2 结果

由于仅有 CPU 版本，没有 CUDA，故本任务略。